МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

Лабораторная работа №4 по курсу

«Операционные системы и архитектура компьютера»

«Работа с файловыми системами. Обеспечение обмена данных между процессами посредством технологии File mapping»

Студент: Цыганок Юлия Дмитриевна
Группа: М8О–205БВ–25
Преподаватель: Филимонов Н. С.
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2026

1. ВВЕДЕНИЕ

1.1 Цель работы

Приобретение практических навыков в освоении принципов работы с файловыми системами и обеспечении обмена данных между процессами посредством технологии «File mapping» отображение файлов в память.

1.2 Задание

Разработать программу на языке Си для многопроцессной обработки строк (Вариант 20). Родительский процесс должен создать два дочерних процесса (отдельные программы) и организовать их взаимодействие через технологию «File mapping». Родитель считывает вводимые пользователем строки и распределяет их: строки длиннее 10 символов передаются второму процессу, остальные – первому. Дочерние процессы инвертируют полученные строки и записывают результат в файлы, имена которых задаются пользователем при запуске. В программе должна быть реализована синхронизация доступа к общей памяти и обработка системных ошибок.

2. ХОД РАБОТЫ

2.1 Описание варианта и схемы взаимодействия

В соответствии с вариантом 20 (представлен на рис. 1) требуется реализовать программу, в которой:

- Родительский процесс запрашивает у пользователя имена двух файлов, создает объект разделяемой памяти и запускает два дочерних процесса;
- Родитель читает строки от пользователя. Если длина строки > 10 символов — она помещается в буфер второй области памяти, иначе — в буфер первой области (для child1);
- child1 ожидает появления данных в первом буфере разделяемой памяти, инвертирует каждую строку и пишет результат в первый файл;
- child2 ожидает появления данных во втором буфере разделяемой памяти, инвертирует каждую строку и пишет результат во второй файл;

- Родитель и дочерние процессы — разные программы, связь между ними обеспечивается через fork() + execve(), а обмен данными — через отображаемый в память файл с флагом MAP_SHARED.

2.2 Используемые системные вызовы

Для реализации поставленной задачи в программе используются стандартные системные вызовы операционной системы Linux, обеспечивающие создание процессов, организацию межпроцессного взаимодействия через каналы и работу с файлами.

shm_open(name, flags, mode)	Создает и открывает объект разделяемой памяти (POSIX shared memory) для доступа нескольких процессов по имени.
ftruncate(fd, length)	Задает размер объекта разделяемой памяти в байтах.
mmap(addr, length, prot, flags, fd, offset)	Отображает объект памяти (или файл) в адресное пространство процесса. Флаг MAP_SHARED делает изменения видимыми для всех процессов.
munmap(addr, length)	Освобождает (отвязывает) область отображенной памяти.
shm_unlink(name)	Удаляет имя объекта разделяемой памяти из системы после завершения работы.
fork()	Создает дочерний процесс (копию родителя).
execve(path, argv, envp)	Заменяет образ памяти процесса новой программой (запускает child1 или child2).
waitpid(pid, &status, 0)	Ожидает завершения дочернего процесса.
open(path, flags, mode)	Открывает/создает файл для записи результатов.

2.3 Методика выполнения работы

Для выполнения работы использовалась операционная система Linux - подсистема WSL 2 в Windows 11, дистрибутив Ubuntu. Компиляция выполнялась компилятором GCC версии 13.

Порядок выполнения работы:

1. Созданы три исходных файла на языке C: parent.c, child1.c, child2.c.
2. Компиляция трёх программ и запуск родительской программы (представлена на рис. 2)

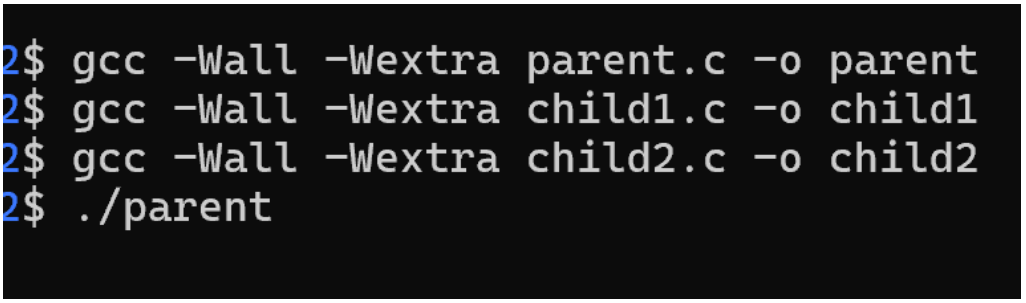


Рис. 2. Компиляция и запуск

3. Ввод имён файлов для child1 и child2.
4. Ввод строк разной длины для проверки фильтрации.
5. Завершение ввода клавишами Ctrl+D.
6. Проверка результатов в файлах через cat.

2.4 Описание работы программы

Код программы и каждого процесса представлен в приложении отчёта по лабораторной работе №4.

1. Родительский процесс:
    a. Запрашивает у пользователя имена двух файлов для сохранения результатов.
    b. Создает объект разделяемой памяти через shm_open() и задает его размер с помощью ftruncate().
    c. Отображает разделяемую память в свое адресное пространство через mmap() с флагом MAP_SHARED и инициализирует структуру SharedData (флаги готовности и буферы).
    d. Через fork() и execve() создает два дочерних процесса, запуская программы ./child1 и ./child2, и передавая им имена файлов в качестве аргументов.
    e. В цикле читает строки из стандартного ввода (stdin):
    f. если длина строки больше 10 символов — ожидает освобождения второго буфера (флаг ready2 == 0), записывает строку в buf2 и устанавливает флаг ready2 = 1;

g. иначе — ожидает освобождения первого буфера (флаг ready1 == 0), записывает строку в buf1 и устанавливает флаг ready1 = 1.
h. По окончании ввода (получение EOF при нажатии Ctrl+D) записывает в переменные len1 и len2 значение -1 (сигнал завершения) и устанавливает флаги ready1 = 1, ready2 = 1.
i. Через waitpid() ожидает завершения обоих дочерних процессов.
j. Освобождает память (munmap) и удаляет объект разделяемой памяти (shm_unlink).

2. Дочерние процессы (child1, child2):
   a. Получают имя файла для записи через аргумент командной строки argv[1].
   b. Открывают файл на запись с помощью open().
   c. Подключаются к уже созданной родителем разделяемой памяти через shm_open() и mmap().
   d. В бесконечном цикле ожидают появления данных: пока соответствующий флаг (ready1 или ready2) равен 0, процесс "спит" с помощью usleep().
   e. Как только флаг становится равен 1, проверяют переменную длины (len). Если она равна -1, цикл прерывается, и процесс завершает работу.
   f. В противном случае дочерний процесс берет строку из своего буфера (buf1 или buf2), инвертирует её (переворачивает) и записывает результат в файл.
   g. После записи сбрасывает флаг готовности в 0, сообщая родителю, что готов принимать новую строку.
   h. По получении сигнала завершения закрывает файл, освобождает память и корректно завершается.

Поскольку механизм mmap не обеспечивает автоматической буферизации и блокировок (в отличие от каналов), в программе используется синхронизация через флаги в разделяемой памяти и функцию usleep(). Это предотвращает состояние гонки (data race), при котором родитель мог бы перезаписать строку до того, как дочерний процесс успел её прочитать.

ВЫВОД

В ходе выполнения лабораторной работы были приобретены практические навыки управления процессами в операционной системе Linux и организации обмена данными между процессами посредством технологии «File mapping».

Была составлена и отлажена программа на языке Си, состоящая из трёх отдельных программ: родительского процесса parent и двух дочерних процессов child1, child2.

Родительский процесс создаёт объект разделяемой памяти через системные вызовы shm_open(), ftruncate() и mmap(), а затем создаёт два дочерних процесса с помощью fork(), после чего заменяет образ памяти каждого ребёнка программой child1/child2 через execve(). Дочерние процессы подключаются к этой же области памяти, что позволяет им и родителю видеть общие данные.

Родительский процесс читает строки, вводимые пользователем, и в зависимости от длины строки помещает их в первый или второй буфер разделяемой памяти в соответствии с правилом фильтрации варианта 20 (порог — 10 символов). Дочерние процессы ожидают появления данных с помощью флагов синхронизации, инвертируют полученные строки и записывают результаты в файлы, имена которых переданы им через аргументы командной строки.

В результате работы были изучены следующие системные вызовы: shm_open, ftruncate, mmap, munmap, shm_unlink, fork, execve, waitpid, open, write, close, usleep. Освоены принципы работы с разделяемой памятью, механизмы синхронизации процессов и обработка системных ошибок через perror(). Все поставленные задачи выполнены в полном объёме, программа работает корректно.

ПРИЛОЖЕНИЕ

Parent.c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <errno.h>

#define MAX_LEN   4096
#define THRESHOLD 10
#define SHM_NAME  "/lab4_shm"
#define SHM_SIZE  65536

typedef struct {
    int ready1;
    int ready2;
    int len1;
    int len2;
    char buf1[MAX_LEN];
    char buf2[MAX_LEN];
} SharedData;

int main(void) {
    char file1[256], file2[256];
```

```
    printf("Файл для child1: ");
    fflush(stdout);
    if (fgets(file1, sizeof(file1), stdin) == NULL) { perror("fgets"); return 1;
}
    file1[strcspn(file1, "\n")] = '\0';

    printf("Файл для child2: ");
    fflush(stdout);
    if (fgets(file2, sizeof(file2), stdin) == NULL) { perror("fgets"); return 1;
}
    file2[strcspn(file2, "\n")] = '\0';

    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd < 0) { perror("shm_open"); return 1; }

    if (ftruncate(shm_fd, SHM_SIZE) < 0) {
        perror("ftruncate");
        close(shm_fd);
        shm_unlink(SHM_NAME);
        return 1;
    }

    SharedData *shm = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shm == MAP_FAILED) {
        perror("mmap");
        close(shm_fd);
        shm_unlink(SHM_NAME);
        return 1;
    }
    close(shm_fd);
```

```
    memset(shm, 0, sizeof(SharedData));


    pid_t pid1 = fork();
    if (pid1 < 0) { perror("fork child1"); return 1; }
    if (pid1 == 0) {
        char *argv[] = {"./child1", file1, NULL};
        char *envp[] = {NULL};
        execve("./child1", argv, envp);
        perror("execve child1");
        _exit(1);
    }


    pid_t pid2 = fork();
    if (pid2 < 0) { perror("fork child2"); return 1; }
    if (pid2 == 0) {
        char *argv[] = {"./child2", file2, NULL};
        char *envp[] = {NULL};
        execve("./child2", argv, envp);
        perror("execve child2");
        _exit(1);
    }


    printf("\nВводите строки:\n");
    char buf[MAX_LEN];
    while (fgets(buf, sizeof(buf), stdin) != NULL) {
        size_t len = strlen(buf);
        int has_nl = (len > 0 && buf[len - 1] == '\n');
        if (has_nl) { buf[len - 1] = '\0'; len--; }


        if ((int)len > THRESHOLD) {
```

```
            while (shm->ready2 != 0) usleep(1000);
            strncpy(shm->buf2, buf, MAX_LEN - 1);
            shm->buf2[MAX_LEN - 1] = '\0';
            shm->len2 = (int)len;
            shm->ready2 = 1;
        } else {
            while (shm->ready1 != 0) usleep(1000);
            strncpy(shm->buf1, buf, MAX_LEN - 1);
            shm->buf1[MAX_LEN - 1] = '\0';
            shm->len1 = (int)len;
            shm->ready1 = 1;
        }
    }

    while (shm->ready1 != 0 || shm->ready2 != 0) usleep(1000);

    shm->len1 = -1; shm->ready1 = 1;
    shm->len2 = -1; shm->ready2 = 1;

    int status;
    waitpid(pid1, &status, 0);
    waitpid(pid2, &status, 0);

    munmap(shm, SHM_SIZE);
    shm_unlink(SHM_NAME);
    return 0;
}
```

Child1.c

```
#define _GNU_SOURCE
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>

#define MAX_LEN  4096
#define SHM_NAME "/lab4_shm"
#define SHM_SIZE 65536

typedef struct {
    int ready1;
    int ready2;
    int len1;
    int len2;
    char buf1[MAX_LEN];
    char buf2[MAX_LEN];
} SharedData;

static void reverse_string(char *s) {
    int n = (int)strlen(s);
    for (int i = 0; i < n / 2; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = t;
    }
}
```

```c
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd < 0) { perror("shm_open child1"); return 1; }

    SharedData *shm = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shm == MAP_FAILED) { perror("mmap child1"); return 1; }
    close(shm_fd);

    while (1) {
        while (shm->ready1 == 0) usleep(1000);

        if (shm->len1 == -1) break;
        reverse_string(shm->buf1);

        if (write(fd, shm->buf1, shm->len1) < 0) { perror("write"); break; }
        if (write(fd, "\n", 1) < 0) { perror("write nl"); break; }

        shm->ready1 = 0;
    }

    close(fd);
    munmap(shm, SHM_SIZE);
    return 0;
}
```

Child2.c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>

#define MAX_LEN  4096
#define SHM_NAME "/lab4_shm"
#define SHM_SIZE 65536

typedef struct {
    int ready1;
    int ready2;
    int len1;
    int len2;
    char buf1[MAX_LEN];
    char buf2[MAX_LEN];
} SharedData;

static void reverse_string(char *s) {
    int n = (int)strlen(s);
    for (int i = 0; i < n / 2; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = t;
    }
```

```c
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }

    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd < 0) { perror("shm_open child2"); return 1; }

    SharedData *shm = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shm == MAP_FAILED) { perror("mmap child2"); return 1; }
    close(shm_fd);

    while (1) {
        while (shm->ready2 == 0) usleep(1000);

        if (shm->len2 == -1) break;

        reverse_string(shm->buf2);

        if (write(fd, shm->buf2, shm->len2) < 0) { perror("write"); break; }
        if (write(fd, "\n", 1) < 0) { perror("write nl"); break; }

        shm->ready2 = 0;
    }
```

```
    close(fd);
    munmap(shm, SHM_SIZE);
    return 0;
}
```